

廈門大學



《汇编语言》实验报告

(四)

姓 名 王子恒

学 号 34520242201240

学 院 信息学院

专 业 软件工程

2025 年 10 月

1.实验目的

1. 熟练使用Debug（更加推荐采用程序形式），理解数据在内存中的存放，并理解并练习各种寻址方式。
2. 学习汇编语言的基本指令，学会阅读汇编代码并动手尝试编写一些算法
3. 学习虚拟机的使用方法，了解BIOS的基础运行方式，尝试破解BIOS密码

2.实验环境

Win11下，DosBox模拟汇编环境

3.实验内容

- 2) 练习使用debug命令破解bios密码，写出自己对破解密码的理解。
- 3) （选做，部分指令知识需要预习）在长度为8的字节数组（无符号数）中，查找大于42H的无符号数的个数，存放在字节单元up中；等于42H的无符号数的个数，存放在字节单元equa中；小于42H的无符号数的个数，存放在字节单元down中。

程序显示up equa down的值。

八个数：31H,21H,42H,52H,87H,23H,98H,01H

4.实验具体实现

- (2) 练习使用debug命令破解bios密码，写出自己对破解密码的理解。

```
D:\>debug
-o 70 10
-i 71
3C
-o 71 ff
-i 71
7F
-q
D:\>
```

破解密码的技术分析

破解计算机密码的过程依赖于对硬件操作，特别是 CMOS 芯片、BIOS 设置以及 I/O 端口的理解。通过这些底层操作，攻击者可以绕过 BIOS 密码验证。以下是关于破解 BIOS 密码的技术要点和操作步骤：

1. 密码存储与硬件交互原理

BIOS 密码存储在主板上的 CMOS 芯片中，CMOS 通过两个 I/O 端口与 CPU 交互：

- **地址端口 (0x70)**：用于选择访问 CMOS 的存储地址。
- **数据端口 (0x71)**：用于读取或写入 CMOS 地址上的数据。

通常，BIOS 密码信息以明文或简单加密方式存储在 CMOS 的特定地址（如 0x30、0x31、0x10）。通过直接操作这些地址，密码信息可以被读取或修改。

2. 破解操作步骤

通过调试工具（如 DOSBox 中的 `debug`），可以执行以下命令来操作 CMOS 芯片，进而破解 BIOS 密码：

- **读取密码：**
 1. `o 70 10`：将 0x10 写入地址端口 0x70，指定访问 CMOS 的地址。
 2. `i 71`：从数据端口 0x71 读取该地址的内容，其中存储了密码信息的一部分。
- **修改密码：**
 1. `o 70 10`：再次指定要操作的 CMOS 地址。
 2. `o 71 ff`：将数据端口写入 0xFF，修改该地址的内容，从而触发 BIOS 将密码重置为无效。

这些命令通过直接操作 I/O 端口，允许读取和修改存储的密码。

3. 不同 CMOS 地址的作用

不同 BIOS 版本和厂商（如 Award、AMI）可能会将密码存储在不同的 CMOS 地址，例如某些系统将密码存储在 0x30、0x31，而其他则可能使用 0x10 地址。操作这些地址时，必须根据具体系统的实现进行调整。

4. 现代 BIOS 安全机制的改进

随着技术的发展，现代 BIOS 密码保护机制已大为改进。相较于早期简单的明文存储方式，现代 BIOS 密码使用了更复杂的加密方式，并对 CMOS 地址的访问进行限制：

- **加密存储**：密码通常被加密存储，直接修改 CMOS 数据不再有效。
- **校验和机制**：修改 CMOS 地址将导致校验和失败，无法成功篡改密码。
- **端口保护**：现代主板限制了对 CMOS 端口（0x70 和 0x71）的访问，防止操作系统直接修改这些端

口。

5. 总结

破解 BIOS 密码的本质是通过操作硬件底层绕过密码验证。在早期系统中，攻击者可以直接修改 CMOS 的值来读取或重置密码。然而，现代系统通过加密、校验和机制和端口保护等手段，加强了密码保护，使得传统的破解方法已经无法应用于现代 BIOS。

(3) (选做，部分指令知识需要预习) 在长度为8的字节数组（无符号数）中，查找大于42H的无符号数的个数，存放在字节单元up中；等于42H的无符号数的个数，存放在字节单元equa中；小于42H的无符号数的个数，存放在字节单元down中。

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
0:\>debug
-a 100
073F:0100 mov ax,31
073F:0103 push ax
073F:0104 mov ax,21
073F:0107 push ax
073F:0108 mov ax,42
073F:010B push ax
073F:010C mov ax,52
073F:010F push ax
073F:0110 mov ax,87
073F:0113 push ax
073F:0114 mov ax,23
073F:0117 push ax
073F:0118 mov ax,98
073F:011B push ax
073F:011C mov ax,1
073F:011F push ax
073F:0120 mov byte ptr [0a00], 0
073F:0125 mov byte ptr [0a01], 0
073F:012A mov byte ptr [0a02], 0
073F:012F mov cx, 8
073F:0132 jmp 140
073F:0134
```

```
-          a 140
073F:0140 pop ax
073F:0141 cmp ax, 42
073F:0144 je 0170
073F:0146 jg 0180
073F:0148 jl 0190
073F:014A jmp 160
073F:014C
-
```

```
-          a 160
073F:0160 dec cx
073F:0161 cmp cx,0
073F:0164 jg 140
073F:0166 call 200
073F:0169 int 3
073F:016A
-
```

```
-a 170
073F:0170 inc byte ptr [0a00]
073F:0174 jmp 160
073F:0176
```

```
-a 180
```

```
073F:0180 inc byte ptr [0a01]
```

```
073F:0184 jmp 160
```

```
073F:0186
```

```
-a 190
```

```
073F:0190 inc byte ptr [0a02]
```

```
073F:0194 jmp 160
```

```
073F:0196
```

```
-          a 200
073F:0200 mov al, [0a00]
073F:0203 call 290
073F:0206 mov dl, 0d
073F:0208 int 21
073F:020A mov dl, 0a
073F:020C int 21
073F:020E mov al, [0a01]
073F:0211 call 290
073F:0214 mov dl, 0d
073F:0216 int 21
073F:0218 mov dl, 0a
073F:021A int 21
073F:021C mov al, [0a02]
073F:021F call 290
073F:0222 mov dl, 0d
073F:0224 int 21
073F:0226 mov dl, 0a
073F:0228 int 21
073F:022A ret
073F:022B
-
```

```

073F:022B
-a 290
073F:0290 add al, 30
073F:0292 mov dl, al
073F:0294 mov ah, 02
073F:0296 int 21
073F:0298 ret
073F:0299

```

```

-g
1
3
4
AX=020A BX=0000 CX=0000 DX=000A SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0169  NU UP EI PL NZ NA PO NC
073F:0169 CC          INT     3

```

运行结果正确

5.实验分析与总结

这次实验主要是通过调试工具（如Debug）来加深对汇编语言和硬件底层操作的理解。通过实践，我们不仅学会了如何用汇编指令操作数据，还尝试了破解BIOS密码，了解了数据存储、寻址方式以及如何通过Debug工具与硬件交互。

1. 破解BIOS密码

- 通过本次实验，我了解了BIOS密码是如何存储在CMOS芯片中的，并且通过Debug命令可以直接操作这些存储地址，从而读取或重置密码。通过写入和读取特定的地址，我们能够绕过BIOS的密码验证。这个过程让我意识到，虽然早期的BIOS密码保护比较简单，但现代系统通过加密存储和校验和机制提升了安全性。

2. 字节数组处理

- 在这个部分，我们需要编写汇编代码来处理一个字节数组，统计大于、等于和小于42H的数目，

并分别存储在不同的字节单元中。通过这道题，我加深了对汇编语言中各种寻址方式和数据操作指令的理解，并且更加熟练地编写和调试汇编程序。

3. 调试工具的应用

- 通过这次实验，我学会了如何使用Debug工具，理解了如何通过调试来查看程序的执行流程和数据变化。这不仅让我更清楚地理解了汇编程序如何执行，也为今后处理更复杂的系统编程问题打下了基础。

总结

总的来说，这次实验让我对汇编语言、内存操作以及调试工具的使用有了更深的理解。通过破解BIOS密码，我体会到了硬件底层的操作原理；而通过处理字节数组，我加强了对汇编指令和数据操作的掌握。虽然现代的BIOS密码保护机制比以前复杂，但我相信这次实验给了我很多宝贵的实践经验，也为以后更深入的学习提供了基础。